



SIGNAL SERIES • JENKINS

# JENKINS COMPLETE

Controller to pipeline to hardening — the full Jenkins craft.

By **Nabawy**

AUTHOR & COMPILER

**Jenkins**

**Topology**

**Setup**

**Jenkinsfiles**

**Agents**

# Table of Contents

|    |                                       |    |    |                              |    |
|----|---------------------------------------|----|----|------------------------------|----|
| 03 | Jenkins                               | 03 | 04 | Rules & Handoff              | 04 |
| 05 | Topology                              | 05 | 06 | Failures & Handoff           | 06 |
| 07 | Setup                                 | 07 | 08 | Platforms                    | 08 |
| 09 | Seven Steps                           | 09 | 10 | Harden & Handoff             | 10 |
| 11 | Jenkinsfiles                          | 11 | 12 | Canonical                    | 12 |
| 13 | DinD & Handoff                        | 13 | 14 | Agents                       | 14 |
| 15 | Agent Wiring                          | 15 | 16 | Groovy Core                  | 16 |
| 17 | Power                                 | 17 | 18 | Idioms                       | 18 |
| 19 | Credentials                           | 19 | 20 | Bindings                     | 20 |
| 21 | Bindings & Handoff                    | 21 | 22 | Plugins                      | 22 |
| 23 | Discipline                            | 23 | 24 | Plugins & Handoff            | 24 |
| 25 | Webhooks                              | 25 | 26 | Webhook Wiring               | 26 |
| 27 | Webhooks & Handoff                    | 27 | 28 | Security                     | 28 |
| 29 | Trust                                 | 29 | 30 | Security & Handoff           | 30 |
| 31 | Advanced                              | 31 | 32 | As Code                      | 32 |
| 33 | Check & Handoff                       | 33 | 34 | Triage                       | 34 |
| 35 | Logs & Recovery                       | 35 | 36 | Quick Revision — Terms Table | 36 |
| 37 | Concept Questions — Distilled Answers | 37 | 38 | Command Cheat Sheet          | 38 |
| 39 | Self-Quiz + Answer Key                | 39 |    |                              |    |

- How to use this book legend
- NOTE · context   KEY · must retain   WATCH OUT · gotcha   VERIFIED · check

**NOTE** Read cover to cover once, then use the Contents to jump. Every chapter ends with a EXAM box — if you can answer it cold, move on.

# Jenkins

Picking up from *Feedback*, where the loop closed — we now *implement that entire loop in Jenkins*, starting with the topology that runs everything.

## 01 / WHY JENKINS

- **WHAT** — open-source automation server: pipelines as code, 1,800+ plugins, controller + agents. Self-hosted, free, runs anywhere.
- **WHY** — your data/secrets on your infra; reaches air-gapped networks and special hardware; no per-minute billing.
- **WITHOUT IT** — releases from laptops: no audit trail, secrets in shell history, one person's memory as infrastructure.

## 02 / CONCEPT MAP

| Generic   | Jenkins                  |
|-----------|--------------------------|
| Pipeline  | Jenkinsfile              |
| Runner    | Agent + executors        |
| Trigger   | Webhook / multibranch    |
| Secrets   | Credentials + bindings   |
| Server    | Controller + agents      |
| Hardening | Auth matrix, agent trust |

Threads: [Controllers & Agents](#)

# Rules & Handoff

## 03 / THE GOLDEN RULE

### DISTINCTIVELY JENKINS ONLY

If a paragraph would be true for GitHub Actions too, it belongs in the generic docs — not here. Jenkins files contain only what is distinctively Jenkins.

## 04 / READING ORDER

**PATH** Architecture (08) → Setup (09) → Pipelines (10) → Groovy (11) → Agents (12) → Credentials (13) → Plugins (14) → Webhooks (15) → Security (16) → Advanced (17) → Troubleshooting (18).

Begin in **Jenkins Architecture** — controller, agents, and executors first, because every later PDF assumes them.

 **EXAM** Jenkins in one breath: an open-source automation server where **pipelines as code** (Jenkinsfiles) run on **agents** under a **controller** that holds config, queue, history and credentials; 1,800+ plugins supply checkout, agents and gates. This book keeps only what is **distinctively Jenkins** and walks Architecture → Setup → Pipelines → Groovy → Agents → Credentials → Plugins → Webhooks → Security → Advanced → Troubleshooting.

Next: [Topology](#)

Threads: [Controllers & Agents](#)

# Topology

Picking up from the *Domain map* — we now examine the *controller–agent topology* that every pipeline, agent, and credential later plugs into.

## 01 / THE THREE PARTS

- **Controller** — holds config (JENKINS\_HOME), queue, history, UI/API, credentials. Runs no builds in production.
- **Agent** — a machine offering **executors**: numbered slots, one build each.
- **Remoting** — the channel shipping work out. Inbound = agent dials out (survives NAT); SSH = controller dials in (flat networks).

## 02 / WHAT LIVES IN JENKINS\_HOME

| Path                       | Holds                                     |
|----------------------------|-------------------------------------------|
| jobs/                      | Job configs + build history               |
| workspace/                 | Checked-out code (rebuildable, deletable) |
| credentials.xml + secrets/ | Encrypted secrets — back up securely      |
| config.xml                 | Global configuration                      |

Threads: [Controllers & Agents](#)

# Failures & Handoff

## 05 / FAILURE MODES

- Disk fills with old builds → UI freezes; fix with build discarders + log rotation.
- Everything pinned to built-in node → single contention point; fix with labeled agents.
- Clock skew → signatures/webhooks fail; run NTP everywhere.

### Q1 Controller vs agent responsibilities?

Controller schedules, serves, stores. Agents execute — nothing else.

## 06 / HANDOFF

Topology understood — now build one. Continue in **Jenkins Setup**.

🎓 **EXAM** Topology in one breath: the **controller** schedules, serves and stores (JENKINS\_HOME: jobs/, workspace/, credentials.xml + secrets/, config.xml) and runs **zero builds**; **agents** offer **executors**, one build per slot, reached by **inbound remoting** (agent dials out, survives NAT) or **SSH** (controller dials in). Classic failures: disk full of old builds, everything pinned to the built-in node, clock skew.

▶ Next: **Setup**

Threads: **Controllers & Agents**

# Setup

Picking up from *Architecture*, where the topology was defined — we now *run it*: one container, one volume, one socket.

## 01 / RUN & UNLOCK

```
$ docker volume create jenkins_home
$ docker run -d --name jenkins \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  jenkins/jenkins:2.516-lts-jdk17
$ docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Open localhost:8080 → paste the password → install suggested plugins → create admin user → set built-in executors to **0**.

**WHEN TO USE JENKINS** use when self-hosted or air-gapped execution is required · use when pipelines span mixed legacy systems · use when plugin ecosystem covers the gap · avoid when greenfield SaaS with no ops team · avoid when zero-maintenance CI is the goal. [Tool vs Tool — The Whole Field](#)

# Platforms

## 02 / WHERE TO RUN IT

| Method          | When                         |
|-----------------|------------------------------|
| Docker image    | Default: dev, training, labs |
| OS package      | Long-lived on-prem server    |
| Kubernetes Helm | Production, ephemeral agents |

**WINDOWS** native msi works, but prefer Docker Desktop (WSL2 backend): same command, Linux-normal paths. Keep a Windows **agent** (label windows) only for .NET/MSBuild workloads. Watch bat vs sh, CRLF, and firewall ports 8080/50000.

## 03 / SMOKE TEST

```
New Item → Pipeline → `hello` → inline script:
pipeline { agent any { stage('Hello') { echo 'alive' } } }
Build Now → green ball = controller + executors + workspace work
```

Threads: [Controllers & Agents](#)

## Seven Steps

### 04 / RUN & USE, END TO END

1 • **Run** — container + volume + socket (above).

2 • **Unlock** — password, suggested plugins, admin user.

3 • **Lock controller** — built-in executors = 0.

4 • **Agent** — new node linux-1, labels linux docker, idle in Executor Status.

5 • **Multibranch job** — repo URL + credentials; Jenkinsfile discovered per branch.

6 • **Webhook** — /github-webhook/; push → build in seconds.

7 • **Verify loop** — green → staging → approve → production. Break it on purpose once.

Threads: [Controllers & Agents](#)

# Harden & Handoff

## 05 / PRODUCTION CHECKLIST

- Persistent, backed-up JENKINS\_HOME (test restore).
- Pinned image + plugin versions; deliberate upgrades.
- TLS reverse proxy + auth matrix from day one.
- Lost data on recreate = volume wasn't mounted. Port 50000 blocked = agents can't connect.

## 06 / HANDOFF

Server running and first job green — now write real pipelines. Continue in **Jenkins Pipelines**.

🎓 **EXAM** Setup in one breath: one volume, one container, one socket — `docker volume create jenkins_home`, run pinned LTS with port 8080/50000, unlock with `initialAdminPassword`, install suggested plugins, then set **built-in executors to 0**; the seven steps run → unlock → lock controller → agent → multibranch job → webhook → verify loop, sealed by a production checklist of backed-up JENKINS\_HOME, pinned versions and TLS plus auth matrix.

▶ Next: [Jenkinsfiles](#)

Threads: [Controllers & Agents](#)

# Jenkinsfiles

Picking up from *Setup*, where the first smoke job went green — we now write *production Jenkinsfiles*: Declarative structure, multibranch, and the full CI→CD arc.

## 01 / GENERIC → JENKINS

| Generic      | Jenkins                                              |
|--------------|------------------------------------------------------|
| Pipeline     | <code>pipeline { ... }</code> in Jenkinsfile         |
| Stage / Step | <code>stage('Build') / sh, echo, plugin steps</code> |
| Runner       | agent directive (label, docker, any)                 |
| Artifact     | <code>archiveArtifacts / pushed image</code>         |

## 02 / DECLARATIVE VS SCRIPTED

**DECLARATIVE** (default, 95%) — opinionated skeleton, syntax-checked pre-run. **SCRIPTED** — raw Groovy (`node { stage(...) }`) for dynamic stages. Language: Apache Groovy; DSL steps documented in the built-in snippet generator.

Threads: [Controllers & Agents](#)

## 03 / CANONICAL JENKINSFILE

```
pipeline {
  agent { label 'linux' }
  stages {
    stage('Build') { steps { sh 'npm ci && npm run build' } }
    stage('Test') { steps { sh 'npm test' }
      post { always { junit 'reports/**/*.xml' } } }
    stage('Image') { steps { sh 'docker build -t registry/app:$GIT_COMMIT .' } }
  }
  post { failure { mail to: 'team@acme.io' } }
}
```

## 04 / MULTIBRANCH = PR GATES

A Multibranch job auto-discovers branches/PRs containing a Jenkinsfile and builds each independently — the Jenkins-native PR gate. Prefer it over hand-wired jobs per branch.

Threads: [Controllers & Agents](#)

# DinD & Handoff

## 06 / SOCKET VS DIND

|           | Socket mount                | DinD                         |
|-----------|-----------------------------|------------------------------|
| Isolation | Weak — shared daemon        | Strong — own daemon          |
| Risk      | Malicious build = host root | Blast radius: dind container |
| Use when  | Trusted teams (fast)        | Untrusted builds             |

- `agent` any everywhere → builds land on controller; use labeled agents.
- Inline scripts → unreviewable; keep Jenkinsfiles in the repo.
- Giant single stage → no visibility; mirror build/test/scan 1:1.

**Q2 Declarative vs Scripted?**  
Declarative for 95% (validated, readable); Scripted for dynamic stages and advanced flow.

## 07 / HANDOFF

Pipelines need a language upgrade next: the Groovy subset that powers them lives in **Groovy Essentials**.

 **EXAM** Jenkinsfiles in one breath: **Declarative** (`pipeline { ... }`) is the default for 95% — validated pre-run and readable — while **Scripted** (`node { stage(...) }`) covers dynamic stages; the canonical shape is Build → Test (with `junit` post) → Image tagged by `$GIT_COMMIT`, and **Multibranch** turns each branch Jenkinsfile into its own PR gate. Antipatterns: `agent any` everywhere, inline scripts, one giant stage, socket mounts for untrusted builds (use DinD).

Next: [Agents](#)

Threads: [Controllers & Agents](#) · [Approval Gates](#)

# Agents

Picking up from *Groovy* — pipelines written, language in hand — we now provision *the machines that execute them*.

## 01 / THREE KINDS

| Type         | Lifecycle           | Best for                        |
|--------------|---------------------|---------------------------------|
| Permanent    | Long-lived VM       | Special HW, Windows/GPU         |
| Docker cloud | Container per build | Default Linux — clean every run |
| Kubernetes   | Pod per build       | Elastic, bursty CI              |

## 02 / LABELS & EXECUTORS

```
agent { label 'linux && docker' } // must have BOTH
agent { docker { image 'node:20' } } // ephemeral container
agent none                        // each stage picks
```

**LABEL** = capability tag matched by jobs. **EXECUTOR** = one build slot (~1 per 1–2 cores). Wrong labels = builds queue forever.

Threads: [Controllers & Agents](#)

# Agent Wiring

## 03 / CONNECTION & TRUST

- **Inbound** — agent dials out via WebSocket; survives NAT, needs port 50000.
- **SSH** — controller dials in; simple on flat networks, needs port 22 + credential.
- **Ephemeral wins** — pristine container per run kills "works on my machine" and tool drift.

### TRUST BOUNDARY

Never run untrusted fork PRs on permanent agents holding deploy credentials. Forks get isolated, credential-free agents — a malicious workflow otherwise owns your box.

- Label typo → queue forever; check Executor Status.
- Too many executors per CPU → starvation.

## 04 / HANDOFF

Agents run the work — next they need secrets. Continue in **Jenkins Credentials**.

 **EXAM** Agents in one breath: **permanent** nodes for special hardware, **Docker cloud** containers per build as the Linux default, **Kubernetes** pods per build for bursty CI — matched by **labels** (capability tags, `linux && docker`) while an **executor** is one build slot sized ~1 per 1–2 cores; **ephemeral wins** because a pristine container kills tool drift, and fork PRs run only on **isolated, credential-free** agents.

Next: **Groovy**

Threads: [Controllers & Agents](#)

# Groovy Core

Picking up from *Pipelines*, where Declarative met Scripted — this is *the Groovy subset both run on*.

## 01 / STRINGS, COLLECTIONS, CONDITIONALS

```
// "double quotes" interpolate, 'single' don't
def tag = env.GIT_COMMIT.take(7)
echo "Building ${app}:${tag}"
stages.each { s -> echo "stage: ${s}" }
def failed = results.findAll { k, v -> v != 'SUCCESS' }
def env2 = env.BRANCH_NAME == 'main' ? 'prod' : 'staging'
```

# Power

## 02 / DYNAMIC STAGES, SAFE NULLS, MAPS

```
// one stage per service – impossible in pure Declarative
['api','web','worker'].each { svc ->
  stage("Test ${svc}") { sh "./test.sh ${svc}" } }
// ?. skips nulls, ?: falls back
def owner = env.CHANGE_AUTHOR?.toLowerCase() ?: 'unknown'
def cfg = [retries: 3, timeout: 20]
timeout(time: cfg.timeout, unit: 'MINUTES') { sh 'make ci' }
```

# Idioms

## 03 / JENKINSFILE IDIOMS

```
withCredentials([string(credentialsId: 'api-token', variable: 'T')]) {  
    sh 'curl -H "Authorization: Bearer $T" ...' }  
retry(3) { sh './flaky-deploy.sh' }  
catchError(buildResult: 'UNSTABLE') { sh './optional-audit.sh' }
```

## 04 / SANDBOX GOTCHAS

| Code                         | Fix                            |
|------------------------------|--------------------------------|
| <code>new File().text</code> | Use <code>readFile</code> step |
| <code>System.getenv()</code> | Use <code>env.X</code>         |
| <code>.execute()</code>      | Use <code>sh</code> step       |

**RULE** if Groovy fights you, push logic into `sh` and keep Groovy as glue. Next: give pipelines somewhere to run — **Jenkins Agents**.

**EXAM** Groovy in one breath: "double quotes" interpolate and 'single' do not; `each/findAll/ternary` shape data, `?.` skips nulls and `?:` falls back; Scripted-only dynamic stages fan one stage per service; pipeline idioms are `withCredentials`, `retry(3)` and `catchError`; the **sandbox** bans `new File`, `System.getenv` and `.execute()` — use `readFile`, `env.X` and `sh`, and push real logic into `sh` keeping Groovy as glue.

Next: [Credentials](#)

Threads: [Controllers & Agents](#)

# Credentials

Picking up from *Agents*, where executors run untrusted build code — secrets must therefore *never live in Jenkinsfiles*. Enter the credentials store.

## 01 / KINDS & SCOPES

| Kind                | Holds                | Binding                             |
|---------------------|----------------------|-------------------------------------|
| Secret text         | API token            | <code>string(...)</code>            |
| Username + password | Registry login       | <code>usernamePassword(...)</code>  |
| SSH key             | Git / agent auth     | <code>sshUserPrivateKey(...)</code> |
| Secret file         | kubeconfig, keystore | <code>file(...)</code>              |

**SCOPES** System (server internals) vs Global/Job (pipelines). Narrowest scope that works. IDs are stable references; values never appear in code, logs, or images.

Threads: [Controllers & Agents](#)

# Bindings

## 02 / BINDING IN PRACTICE

```
withCredentials([usernamePassword(
  credentialsId: 'registry-creds',
  usernameVariable: 'U', passwordVariable: 'P')]) {
  sh 'echo "$P" | docker login registry -u "$U" --password-stdin' }
# Jenkins masks $U/$P in console output automatically
```

## 03 / FAILURE MODES

- Secret echoed or baked into an image layer → treat as compromised, **rotate immediately**.
- Wrong scope (System-only in pipeline) → "credentials not found"; move to Global.
- Duplicate IDs → ambiguous resolution; keep IDs unique (github-acme-app-token).

### LEAK = ROTATE

Masking is a backstop, not a guarantee. Any secret that touched a log, layer, or chat is compromised — replace by ID, zero Jenkinsfile changes.

# Bindings & Handoff

## 04 / HANDOFF

Secrets stored — but who delivers all these capabilities? Continue in **Jenkins Plugins**, where Git, Docker, and credentials bindings themselves come from.

🎓 **EXAM** Credentials in one breath: kinds are **secret text**, **username + password**, **SSH key** and **secret file**, scoped **System** (server internals) versus **Global/Job** (pipelines) with the narrowest scope that works and unique stable IDs; `withCredentials` binds values to masked variables so secrets never live in Jenkinsfiles, logs or image layers — and **LEAK = ROTATE**: anything that touched a log, layer or chat is compromised and replaced by ID with zero Jenkinsfile changes.

▶ Next: [Plugins](#)

Threads: [Controllers & Agents](#)

# Plugins

Picking up from *Credentials* — whose bindings are themselves a plugin — we now survey *the ecosystem that makes Jenkins Jenkins*.

## 01 / ESSENTIAL SET

| Plugin                 | Why                               |
|------------------------|-----------------------------------|
| Pipeline (workflow-*)  | Declarative/Scripted, stage/steps |
| Git + Branch Source    | Checkout, multibranch, webhooks   |
| Credentials (+Binding) | Secret storage + withCredentials  |
| Docker / Kubernetes    | Ephemeral build agents            |
| JUnit / Warnings NG    | Test reports, analysis gates      |

Threads: [Controllers & Agents](#)

# Discipline

## 02 / PIN & STAGE UPGRADES

```
# plugins.txt – pinned, version-controlled
git:5.2.1
workflow-aggregator:596.v8c21c963d92d
credentials-binding:681.v346b_f30f5d80
docker-workflow:572.v950f58993843
```

**RULES** upgrade in a staging Jenkins first (plugin updates, not core, cause most outages) · remove unused plugins (attack surface + startup + conflicts).

## 03 / FAILURE MODES

- Update-all Friday → Monday outage; update deliberately with a snapshot.
- No such DSL method → owning plugin not installed.
- Core/plugin skew → cryptic startup failures; keep LTS + compatible set.

### FRIDAY RULE

Never bulk-update plugins before a weekend. Deliberate upgrades with a rollback snapshot, or Monday owns you.

Threads: [Controllers & Agents](#)

# Plugins & Handoff

## 04 / HANDOFF

Plugins installed — now connect the trigger wire. Continue in **Jenkins Webhooks**, where pushes become builds in seconds.

🎓 **EXAM** Plugins in one breath: the essential set is **Pipeline (workflow-\*)**, **Git + Branch Source**, **Credentials (+Binding)**, **Docker/Kubernetes** and **JUnit/Warnings NG** — pinned in a version-controlled `plugins.txt`, upgraded in a **staging Jenkins first** with unused plugins removed; **Friday rule**: never bulk-update before a weekend, keep a snapshot, because plugin updates (not core) cause most outages and No such DSL method always means the owning plugin is missing.

➡ Next: [Webhooks](#)

# Webhooks

Picking up from *Plugins*, which delivered the Branch Source plugin — we now *wire it*: webhooks that turn pushes into builds.

## 01 / TRIGGER OPTIONS

| Method                | Latency | Verdict               |
|-----------------------|---------|-----------------------|
| Webhook               | Seconds | Default               |
| Multibranch + webhook | Seconds | Default for PRs       |
| SCM polling           | Minutes | Fallback / air-gapped |
| Scheduled / manual    | —       | Nightly, promotions   |

# Webhook Wiring

## 02 / GITHUB WEBHOOK SETUP

1. Job: enable GitHub hook trigger for GITScm polling
2. Repo → Settings → Webhooks → Add:  
Payload URL `https://jenkins.acme.io/github-webhook/`  
events: `push + pull requests`
3. Push → GitHub Hook Log shows delivery → build in seconds  
# private nets: controller needs a reachable URL (reverse proxy / ngrok)

## 03 / MULTIBRANCH = PR GATES

Multibranch scans the repo, builds each branch/PR Jenkinsfile independently, reports status to GitHub checks. Branch deleted → job auto-removed. No hand-wired jobs per branch, ever.

- 403/404 → wrong URL (trailing slash matters) or missing plugin.
- Not triggering → trigger checkbox off; check Hook Log.
- Duplicates → webhook + polling both on; disable polling.
- Validate webhook secrets — or anyone curls your builds.

Threads: [Controllers & Agents](#)

# Webhooks & Handoff

## 04 / HANDOFF

Triggers fire — but open triggers are an attack surface. Continue in **Jenkins Security**, where authentication, authorization, and agent trust lock this down.

🎓 **EXAM** Webhooks in one breath: **webhook** (seconds) is the default trigger with polling as fallback and schedule/manual for nightlies and promotions — the GitHub hook fires at `/github-webhook/` (trailing slash matters) and **Multibranch** scans the repo, builds each branch/PR Jenkinsfile independently, reports GitHub checks and auto-removes dead branches; validate webhook secrets so nobody curls your builds, and never run webhook plus polling together.

▶ Next: [Security](#)

Threads: [Controllers & Agents](#)

# Security

Picking up from *Webhooks*, where anyone-with-curl could fire builds — we now decide *who may do what*.

## 01 / AUTHENTICATION & AUTHORIZATION

| Role       | Grants               |
|------------|----------------------|
| Developers | Job/Build + Read     |
| Leads      | + Configure / Cancel |
| Admins     | Overall / Administer |

**RULES** SSO (Google/GitHub/Okta) for teams, local users for labs · start from "logged-in users can do nothing", grant upward · audit everything with the Audit Trail plugin.

# Trust

## 02 / AGENT-TO-CONTROLLER DISTRUST

- Builds execute attacker-influenced code — default deny on Agent → Controller access.
- Ephemeral agents: a compromised container dies with the build.
- Fork PRs → isolated, credential-free agents. Never shared permanent ones.

## 03 / SECRETS & UPDATES

- Secrets only via bindings; set +x around manual handling; TLS-only admin.
- Rotate by ID replacement; deprovision SSO, don't rely on memory.
- CVEs cluster in plugins: staging validates → snapshot → production.

### IF SECRETS LEAK

JENKINS\_HOME/secrets decrypts the whole store. Its exposure = everything is compromised. Rotate all of it.

### Q3 Why are PR builds a trust boundary?

PR contents are attacker-influenced code running on your agents — isolate execution and scope credentials.

Threads: [Controllers & Agents](#)

# Security & Handoff

## 04 / HANDOFF

Hardened core — now scale the craft. Continue in **Jenkins Advanced**: shared libraries, matrix builds, JCasC, and backup/restore.

🎓 **EXAM** Security in one breath: **SSO** (Google/GitHub/Okta) for teams and local users for labs, a deny-by-default **matrix** (developers build, leads configure, admins administer) with **Audit Trail** on everything; **agent** → **controller access is default-deny** because builds run attacker-influenced code — ephemeral agents die with the build, fork PRs stay isolated and credential-free, secrets travel only via bindings, and exposure of `JENKINS_HOME/secrets` means rotating everything since CVEs cluster in plugins.

➡ Next: [Advanced](#)

Threads: [Controllers & Agents](#)

## Advanced

Picking up from *Security*, where trust boundaries were drawn — shared libraries run *outside the sandbox*, so that trust model governs them.

### 01 / SHARED LIBRARIES

```
// Jenkinsfile: one line replaces 60 of boilerplate
@Library('acme-pipeline-lib@1.4') _
standardServicePipeline(app: 'shop', deployEnv: 'staging')
```

**RULES** version the lib (@1.4, never @main) · test with JenkinsPipelineUnit before tagging · only admins approve library repos (unsandboxed).

### 02 / PARALLEL & MATRIX

Parallelize **independent verification** (unit/lint/scan), never dependent deploys. Matrix multiplies stages × axes (JDK 17/21, browsers). Size agent pools for **peak fan-out**, not average load.

Threads: [Controllers & Agents](#)

# As Code

## 03 / CONTROLLER FROM YAML

```
# jenkins.yaml – identical controller every boot
jenkins: { numExecutors: 0 }
credentials: { system: { ... registry-token: "${REGISTRY_TOKEN}" } }
# boot: pinned image + plugins.txt + jenkins.yaml. UI drift is ephemeral.
```

## 04 / BACKUP, CONSOLE, TUNING

- **Backup** JENKINS\_HOME minus workspaces; restore jobs + credentials + config + YAML + plugins. Untested backup = rumor. Encrypt off-site.
- **Script Console** = Groovy as admin; audit every run; never paste blind snippets.
- **Queue** — executors  $\leq \sim 1$  per 1–2 cores; heap  $\geq 4$  GB ( $\sim Xmx4g$ ); stagger cron (H 9 \*); quiet periods batch rapid commits.

**PROD TOPOLOGY** active-passive controllers from one jenkins.yaml + replicated storage · agent fleets per team/env (labels route) · controller private behind TLS proxy, agents outbound-only · quarterly DR restore drill measured against RTO.

Threads: [Controllers & Agents](#)

# Check & Handoff

## 05 / INTERVIEW CHECK

### Q4 Roll out a shared library to 50 repos?

Version + unit-test the lib, migrate one pilot repo, pin tags, admin-approve the repo (unsandboxed trust).

### Q5 Rebuild-a-controller drill order?

Image → plugins → JCasC → secrets → backup restore → smoke pipeline. Secrets break first.

## 06 / HANDOFF

Advanced patterns banked — when they break, diagnose in order. Continue in **Jenkins Troubleshooting**.

🎓 **EXAM** Advanced in one breath: one versioned line — `@Library('acme-pipeline-lib@1.4')` `_`, never `@main`, unit-tested with `JenkinsPipelineUnit` and admin-approved since libraries run unsandboxed — replaces 60 lines of boilerplate; **parallel** multiplies independent verification and **matrix** multiplies stages by axes, sized for peak fan-out; `jenkins.yaml` (**JCasC**) makes the controller reproducible, `JENKINS_HOME` minus workspaces is the backup (restore tested, off-site, encrypted), and prod runs active-passive controllers with a quarterly DR drill against RTO.

▶ Next: [Troubleshooting](#)

Threads: [Supply-Chain Security](#) · [Controllers & Agents](#)

# Triage

Picking up from *Advanced*, where moving parts multiplied — this is *the diagnostic order that untangles them*.

## 01 / TRIAGE TABLE

| Symptom            | Check                           | Fix                           |
|--------------------|---------------------------------|-------------------------------|
| Stuck in queue     | Executor Status, labels, agents | Fix typo, free executor       |
| No such DSL method | Owning plugin installed?        | Install / pin plugin          |
| command not found  | Tool on that agent?             | Matching label / Docker image |
| Clone/auth failure | Credential ID + scope           | Rebind Global credential      |
| Webhook silent     | Hook Log, checkbox              | Fix URL, enable trigger       |
| Controller frozen  | Disk, heap (-Xmx4g)             | Discard builds, raise heap    |

Threads: [Controllers & Agents](#)

# Logs & Recovery

## 02 / READING A BUILD LOG

- First red line = **effect**; scroll up for the **cause** (exit code, Groovy stack).
- **Replay** button re-runs edited pipelines without pushing commits.
- Wipe workspace + rebuild separates dirty-workspace flakes from real failures.

## 03 / SANDBOX & RECOVERY

**SANDBOX** untrusted scripts run allowlisted only; beyond that needs Script Approval. Prefer allowlisted steps — each approval is future maintenance.

**RECOVERY** — bad config? Restore `config.xml` + Reload Configuration (no restart). Disaster? Restore `JENKINS_HOME` backup to a fresh pinned image, replay the smoke pipeline.

## 04 / HANDOFF

Jenkins domain complete — now meet the sibling implementation. Continue in **GitHub Actions**: hosted runners, YAML, zero infrastructure.

🎓 **EXAM** Troubleshooting in one breath: diagnose in order — stuck in queue → labels/executors, No such DSL method → owning plugin, command not found → tool on that agent, clone failures → credential scope, silent webhooks → hook log, frozen controller → disk/heap; the **first red line is the effect, scroll up for the cause**, replay without pushing, wipe the workspace to separate flakes from failures, prefer allowlisted steps over approvals, and recover via `config.xml` + Reload or a full `JENKINS_HOME` restore onto a pinned image plus the smoke pipeline.

Threads: [Controllers & Agents](#)



## Quick Revision — Terms Table

|                     |                                                                                                   |
|---------------------|---------------------------------------------------------------------------------------------------|
| <b>Controller</b>   | Schedules, serves UI/API, stores config, history and credentials. Runs zero builds in production. |
| <b>Agent</b>        | A machine offering executors that run builds. Permanent, Docker cloud, or Kubernetes.             |
| <b>Executor</b>     | One numbered build slot on an agent, roughly one per 1–2 cores.                                   |
| <b>Remoting</b>     | Channel shipping work to agents. Inbound = agent dials out; SSH = controller dials in.            |
| <b>JENKINS_HOME</b> | Controller data: jobs/, workspace/, credentials.xml + secrets/, config.xml. Back up securely.     |
| <b>Jenkinsfile</b>  | Pipeline as code in the repo. Declarative pipeline { ... } or Scripted node { ... }.              |
| <b>Declarative</b>  | Opinionated validated pipeline syntax. Default for 95% of Jenkinsfiles.                           |
| <b>Scripted</b>     | Raw Groovy pipelines for dynamic stages and advanced flow.                                        |
| <b>Multibranch</b>  | Job discovering every branch/PR Jenkinsfile and building each independently.                      |

  

|                          |                                                                                              |
|--------------------------|----------------------------------------------------------------------------------------------|
| <b>Label</b>             | Capability tag matching jobs to agents, e.g. linux && docker. Typos queue forever.           |
| <b>Credentials store</b> | Encrypted vault keyed by stable IDs. Values never appear in code or logs.                    |
| <b>withCredentials</b>   | Binding step exposing secrets as masked variables inside sh blocks.                          |
| <b>Shared library</b>    | Versioned Groovy lib (@1.4, never @main) replacing boilerplate. Unsandboxed, admin-approved. |
| <b>JCasC</b>             | Controller from YAML: identical controller every boot from jenkins.yaml.                     |
| <b>Plugin</b>            | Capability unit. Pin in plugins.txt, upgrade in staging first, remove unused ones.           |
| <b>Webhook</b>           | /github-webhook/ trigger turning pushes into builds in seconds.                              |
| <b>Sandbox</b>           | Allowlist restricting untrusted Groovy. Prefer allowlisted steps over approvals.             |
| <b>LTS</b>               | Long-term-support line cut every ~4 weeks. Production default over weeklies.                 |

**KEY IDEA** Cover the right column and define each term aloud. Any stumble goes back to its chapter.

Threads: [Controllers & Agents](#) · [Supply-Chain Security](#)



## Concept Questions — Distilled Answers

### Q1 Controller vs agent responsibilities?

Controller schedules, serves and stores. Agents execute — nothing else.

### Q2 Declarative vs Scripted — when each?

Declarative for 95%: validated, readable. Scripted for dynamic stages and advanced flow.

### Q3 Label vs executor?

Label is a capability tag matching jobs to agents. Executor is one build slot.

### Q4 Why set built-in executors to 0?

So no build ever lands on the controller — it is a single contention point, not a worker.

### Q5 A secret touched a log. What now?

Treat as compromised and rotate by ID immediately. Masking is a backstop, not a guarantee.

### Q6 Multibranch vs hand-wired jobs?

Multibranch auto-discovers branches/PRs, gates each, reports checks, removes dead branches. Never hand-wire.

### Q7 Socket mount vs DinD?

Socket is fast for trusted teams but shares the daemon. DinD isolates untrusted builds.

### Q8 Rebuild-a-controller drill order?

Pinned image → plugins → JCasC → secrets → backup restore → smoke pipeline. Secrets break first.

Threads: [Controllers & Agents](#)



# Command Cheat Sheet

## Docker

```
docker volume create jenkins_home
```

persistent controller data

```
docker run -d -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home jenkins/jenkins:2.516-lts-jdk17
```

pinned LTS controller

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

first unlock password

## Jenkinsfile

```
pipeline { agent { label 'linux' } }
```

labeled agent, never any

```
stage('Test') { steps { sh 'npm test' } }
```

mirrored build/test/scan stages

```
post { always { junit 'reports/**/*.xml' } }
```

test report gate

```
withCredentials([string(credentialsId: 'api-token', variable: 'T')])
```

masked secret binding

```
retry(3) { sh './flaky-deploy.sh' }
```

transient-failure retry

```
@Library('acme-pipeline-lib@1.4') _
```

versioned shared library

**NOTE** Diagnostic order: queue first (labels, executors), then plugin, tool, credential scope, hook log, disk and heap. Never bulk-update on Fridays.

Threads: [Controllers & Agents](#)



## Self-Quiz + Answer Key

Q1 What four paths live in JENKINS\_HOME?

Q2 Inbound vs SSH agent connection?

Q3 The seven setup steps in order?

Q4 What does Multibranch give you?

Q5 Sandbox bans which three calls, and their fixes?

Q6 Friday rule?

Q7 Developers vs leads vs admins?

Q8 First red line: effect or cause?

### ▪ Answer key

1. jobs/, workspace/, credentials.xml + secrets/, config.xml.
2. Inbound: agent dials out, survives NAT. SSH: controller dials in on flat networks.
3. Run, unlock, lock controller, agent, multibranch job, webhook, verify loop.
4. Per-branch/PR builds, GitHub checks, auto-removal of dead branches.
5. new File → readFile; System.getenv → env.X; .execute() → sh.
6. Never bulk-update plugins before a weekend. Stage first, keep a snapshot.
7. Developers build, leads configure/cancel, admins administer. Deny by default.
8. Effect — scroll up for the cause, then replay or wipe the workspace.

**FINAL CHECK** Eight for eight from memory, twice in a row — the Jenkins craft is yours. Open GitHub Actions next.

Threads: [Controllers & Agents](#)